Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №41

ТЕМА: (Нова) Структуровані файли: зчитування та збереження даних у форматі .json.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- ознайомитися з форматом JSON та його призначенням для зберігання структурованих даних;
- вивчити модуль json та його основні функції для роботи з файлами;
- навчитися зберігати словники та списки Python у файл формату .json;
- навчитися зчитувати структуровані дані з .json-файлу назад у програму.

Теоретичні відомості

1. Що таке формат JSON

JSON (JavaScript Object Notation) — це текстовий формат для зберігання структурованих даних, який широко використовується для обміну даними між програмами, серверами й базами даних. Структура JSON дуже нагадує словники та списки Python: дані записуються у вигляді пар «ключ: значення», можуть бути вкладеними одне в одне.

```
{
   "ім'я": "Олена",
   "вік": 20,
   "предмети": ["математика", "python", "фізика"]
}
```

Завдяки такій схожості словник чи список Python дуже легко перетворити у формат JSON і навпаки — саме для цього призначений стандартний модуль json.

2. Збереження даних у файл .json: json.dump()

Функція json.dump(об'єкт, файл) записує словник (або список) у відкритий файл у форматі JSON:

```
import json

student = {
   "ім'я": 'Олена',
   'вік': 20,
   'предмети': ['математика', 'python', 'фізика']
```

```
}

with open('student.json', 'w', encoding='utf-8') as f:
    json.dump(student, f, ensure_ascii=False, indent=4)
```

Параметр ensure_ascii=False потрібен, щоб кириличні символи зберігалися у файлі як звичайний текст, а не як спеціальні код-послідовності; параметр indent=4 робить файл зручним для читання людиною, додаючи відступи.

3. Зчитування даних з файлу .json: json.load()

Функція json.load(файл) виконує обернену дію: зчитує вміст JSON-файлу і перетворює його назад на відповідний об'єкт Python (словник або список):

```
import json

with open('student.json', 'r', encoding='utf-8') as f:
    student = json.load(f)

print(student)
print(student["ім'я"])
print(student['предмети'])
```

Результат: *{"ім'я": 'Олена', 'вік': 20, 'предмети': ['математика', 'python', 'фізика']}*
Олена
['математика', 'python', 'фізика']

Після зчитування student — це звичайний словник Python, з яким можна працювати всіма вже відомими способами: звертатися за ключами, перебирати циклом, змінювати значення тощо.

4. Збереження списку словників (декількох записів)

JSON зручно використовувати для зберігання не одного об'єкта, а цілого списку однотипних записів — наприклад, списку студентів:

```
import json

students = [
    {"ім'я": 'Іван', 'вік': 19},
    {"ім'я": 'Оксана', 'вік': 20}
]

with open('students.json', 'w', encoding='utf-8') as f:
```

```
    json.dump(students, f, ensure_ascii=False, indent=4)

with open('students.json', 'r', encoding='utf-8') as f:
    loaded = json.load(f)

for student in loaded:
    print(student["ім'я"], '-', student['вік'], 'років')
```

Результат: *Іван - 19 років*
Оксана - 20 років

5. Порівняння звичайного текстового файлу і JSON

На відміну від звичайного текстового файлу, де структуру даних (де закінчується одне поле і починається інше) доводиться вигадувати й розбирати самостійно (наприклад, через роздільники типу «;»), формат JSON вже має стандартну, однозначну структуру, яку модуль json розбирає автоматично — це робить його значно зручнішим для зберігання складних, вкладених даних.

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Ціни товарів у завданні 3 залежать від N:

Завдання	Дані варіанта	Приклад N = 7
3	ціни: зошит 10 + N, ручка 5 + N, маркер 20 + N; кількість 5, 10, 3	ціни 17, 12, 27; вартість 286

б) Кожне завдання — окремий файл Прізвище_ПР41_зN.py (наприклад, Ivanenko_PR41_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Файли зберігайте функцією json.dump із ensure_ascii=False та encoding='utf-8', зчитуйте json.load(). У завданні 6* перевіряйте наявність файлу (try/except FileNotFoundError або os.path.exists). Файли .json здавайте разом із програмою.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР41_з1.py, підключіть json. Створіть словник про себе (ім'я, вік, група, список улюблених предметів) і збережіть у profile.json (json.dump, ensure_ascii=False, indent=4, encoding='utf-8').

   Приклад: у файлі видно кирилицю та відступи; поле «улюблені предмети» — список.

2. Зчитайте profile.json і виведіть усі поля у форматі «ключ: значення».

   Приклад: «ім'я: Олег», «вік: 17», … «улюблені предмети: ['інформатика', 'математика']».

3. Збережіть у products.json список із трьох словників товарів вашого варіанта (name, price, qty).

   Приклад (N = 7): [{'name': 'зошит', 'price': 17, 'qty': 5}, {'name': 'ручка', 'price': 12, 'qty': 10}, {'name': 'маркер', 'price': 27, 'qty': 3}].

4. Зчитайте products.json і виведіть «Загальна вартість: … грн» (сума price × qty; два знаки).

   Приклад (N = 7): «Загальна вартість: 286.00 грн».

5. Додайте новий запис до наявного products.json: зчитайте список, виконайте append() і збережіть знову. Новий запис: {'name': 'лінійка', 'price': 15, 'qty': 2}; виведіть нову загальну вартість.

   Приклад (N = 7): «Загальна вартість: 316.00 грн»; у файлі 4 записи.

6. Завдання підвищеної складності*. «Нотатки»: при запуску програма завантажує список із notes.json (якщо файлу немає — порожній список), пропонує додати нотатку, зберігає список і виводить усі нотатки з номерами.

   Приклад: перший запуск: додано «Купити хліб» → «1. Купити хліб»; другий запуск: додано «Здати ПР» → «1. Купити хліб», «2. Здати ПР».

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, кирилиця збережена кодами \uXXXX, не вказано encoding, файл перезаписано замість додавання запису, не оброблено відсутній notes.json) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, додати нове поле чи змінити структуру запису). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Що таке формат JSON і для чого він застосовується?
2. Яка функція модуля json використовується для збереження даних у файл, а яка — для зчитування?
3. Навіщо потрібен параметр ensure_ascii=False при збереженні даних із кириличним текстом?
4. Яким типом даних Python представлений результат json.load(), якщо у файлі збережено список записів?
5. У чому перевага JSON порівняно зі звичайним текстовим файлом для зберігання структурованих даних?